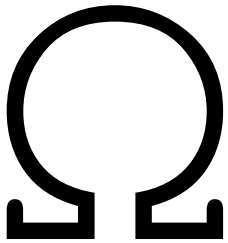

Team Omega

https://teamomega.eth.limo

Almagest Labs iTRY NAV Oracle Integration

Final Audit Report

April 8, 2026

Summary

Almagest Labs/Inverter has asked Team Omega to audit an update of the iTRY contacts with a Redstone Oracle Integration.

The current document is a preliminary audit report. The Inverter team will read this and fix any issues they choose to fix, and Omega will subsequently review the fixes and write a final report.

Team Omega
Team Omega (https://teamomega.eth.limo/) specializes in Smart Contract security audits on the Ethereum ecosystem.

Almagest/Inverter
Inverter (https://www.inverter.network/) is the pioneering Web3 protocol for token economies enabling conditional token issuance, dynamic utility management and token distribution.

Scope of the Audit
The scope of the current audit are the changes made in the following PR:

```
https://github.com/InverterNetwork/iTry-contracts/pull/47
```

Which, at the time that we reviewed the PR, is a Pull request to merge commit:

```
07de603ab50a0d8e39060a8938be64aa8a87bc60
```

Into main, which is at commit:

```
0bc6ac9e1b272254dcc39b55a26ede44f14bea35
```

The PR adds a provider-agnostic NAV oracle abstraction with a three-zone staleness model, implements a Redstone-backed NAV oracle, and updates the existing manual oracle + deployment/test stack to support keeper/threshold configuration.

Methodology

The audit report has been compiled on the basis of the findings from different auditors (Jelle and Ben). The auditors work independently. Each of the auditors has several years of experience in developing smart contracts and web3 applications.

Liability

The audit is on a best-effort basis. In particular, the audit does not represent any guarantee that the software is free of bugs or other issues, nor is it an endorsement by Team Omega of any of the functionality implemented by the software.

Disclaimer

The audit makes no statements or warranties about utility of the code, safety of the code, suitability of the business model, regulatory regime for the business model, or any other statements about fitness of the contracts to purpose, or their bug free status. The audit documentation is for discussion purposes only.

Severity definitions

High	Vulnerabilities that can lead to loss of assets or data manipulations.
Medium	Vulnerabilities that are essential to fix, but that do not lead to assets loss or data manipulations
Low	Issues that do not represent direct exploit, such as poor implementations, deviations from best practice, high gas costs, etc
Info	Matters of opinion

Summary of findings

We found **two medium severity issues** - these are issues that we believe are essential to fix. We found no other issues.

Severity	Number of issues	Number of resolved issues
High		-
Medium	2	-
Low		-
Info		-

Resolution

We delivered a preliminary audit report on April 3, 2026. The inverter team subsequently addressed the issues in a new commit:

`fbbd0f45a311ce1d1286e8aa0e6ce8ab8fcda263`

Both issues we found were fixed in this commit.

Findings

General

G1. On-chain timestamp is not validated [medium] [resolved]

The contract reads `latestRoundData()` but ignores the `updatedAt` field, which indicates when data was last pushed to the blockchain. You're only validating the NAV publication timestamp (from a separate feed).

This means that if the Redstone relayer fails to update the contract's price after a new NAV price was published, this will be detected up to 72 hours later. Or, in other words, checking the NAV publication timestamp will guarantee that the returned price was published less than 72 hours ago, but, when Redstone relayers fail, it does not guarantee that a new price was not published in the meantime.

The failure scenario looks like this:

1. At time t0, NAV is published and immediately pushed on-chain by Redstone
2. At time t1, a new NAV was published, but redstone fails to push the data on-chain
3. As long as the NAV publication date t0 is not expired, the RedstoneNavFeed contract will return the price from t0.

Recommendation: Also check the `updatedAt` value with a much smaller time interval than 72 hours. You could also add some sanity checks on the return price (you are already checking that the price is not equal to 0, but you can also check that it is not unreasonably high, e.g. require that the price is less than 20% of the previous price).

Resolution: the issue was resolved as recommended

G2. Data is read from two independent feeds without checking correlation [medium] [resolved]

The RedstoneNAVFeed contract reads the price from one contract and timestamp from another, with no verification they correspond to the same data point. If the relayer updates them at different times (which could also happen if the relayer updates the values in two separate transactions which are mined at different times), you could get mismatches between the two.

For example, you could get Thursday's price with Friday's timestamp, and erroneously conclude that the price you are reporting is not stale.

Recommendation: Using the `roundId` from the response will not work, as the Redstone contracts always return a value of 1. Adding a tolerance check that `updatedAt` timestamps are within N seconds of each other.

-

Resolution: the issue was resolved as recommended